Cher Naïck TD2B

Rapport Individuel

SAE 4.01 – Développement d’une application complexe

Lors de cette SAE réalisée en groupe avec Clément MOISAN et GUEDON Anaëlle, dans laquelle nous devions réaliser une application gestion de vols et d’aéroports, en lien avec une API, j’ai pu mettre en œuvre de nombreux apprentissages critiques :

-**AC21.01 | Elaborer et implémenter les spécifications fonctionnelles et non fonctionnelles à partir des exigences ; AC21.03 | Adopter de bonnes pratiques de conception et de programmation ; AC22.02 | Utiliser des techniques algorithmiques adaptées pour des problèmes complexes ; AC23.01 | Concevoir et développer des applications communicantes ; AC23.02 : Utiliser des serveurs et des services réseaux virtualisés ; AC24.04 | Manipuler des données hétérogènes :** Dans notre application mobile, réalisée en Flutter avec l’aide d’Anaëlle, nous avons pu implémenter des méthodes consommant notre API en envoyant des requêtes GET pour en récupérer les données de différents types (Aéroport et Vol) (**annexe 1**). Nous avons pu utiliser ces données récupérées pour obtenir des correspondances en liant les vols entre eux de façon efficace (**annexe 2**).

AC21.02 | Appliquer des principes d’accessibilité et d’ergonomie : Notre objectif durant le développement de l’application était de la rendre simple d’utilisation, en affichant les informations souhaitées rapidement (**annexe 3**).

AC21.03 | Adopter de bonnes pratiques de conception et de programmation ; AC22.02 | Utiliser des techniques algorithmiques adaptées pour des problèmes complexes ; AC24.04 | Manipuler des données hétérogènes : La base de données, utilisée par notre API, notre SPA et notre Flutter a été conçue et réalisée avec l’aide d’Anaëlle. Nous avions d’abord construit un modèle MCD (**annexe 4**) permettant de séparer les données indiquées dans le sujet, en plus de simplifier l’implémentation en SQL, de plus nous avions réalisé des requêtes montrant que notre modèle est fonctionnel et adapté à notre projet (**annexe 5**).

AC26.02 | Appliquer une démarche pour intégrer une équipe informatique au sein d’une organisation ; AC26.04 | Rendre compte de son activité professionnelle : En tant que chef du groupe, j’ai pu guider mon équipe grâce à Github (**annexe 6**), permettant à chacun de travailler sur sa partie du projet et de s’organiser, et grâce à Discord (**annexe 7**) où nous avions discuter d’améliorations possibles et où nous avions pu communiquer sur l’avancée de chacun. De plus j’ai produit le document indiquant le suivi du projet.

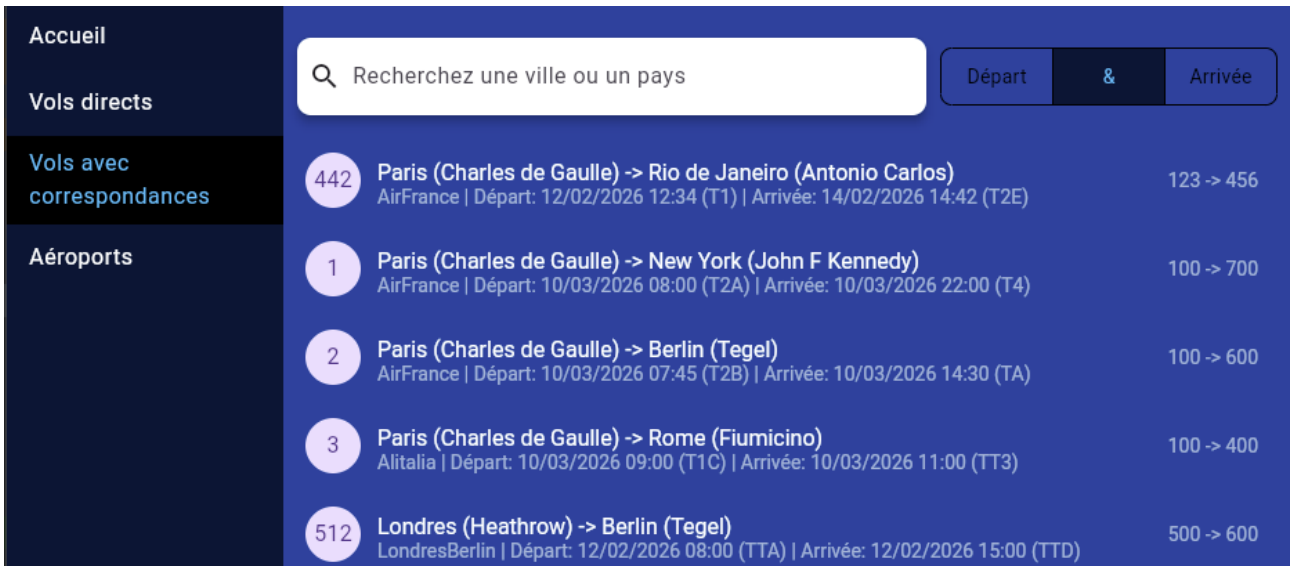
Annexes :

```
// Récupère les aéroports
Future<List<Aeroport>> getAeroports() async{
  final response = await http.get(
    Uri.parse('${Config.apiUrl}/api/Aeroports'),
    headers: {'Accept' : 'application/json'},
  );

  if(response.statusCode == 200){
    final List<dynamic> json = jsonDecode(response.body);
    final aeroports = <Aeroport> [];
    for(var aeroport in json){
      aeroports.add(Aeroport.fromJson(aeroport));
    }
    return aeroports;
  }else{
    throw Exception('Ca a pas marché :(');
  }
}
```

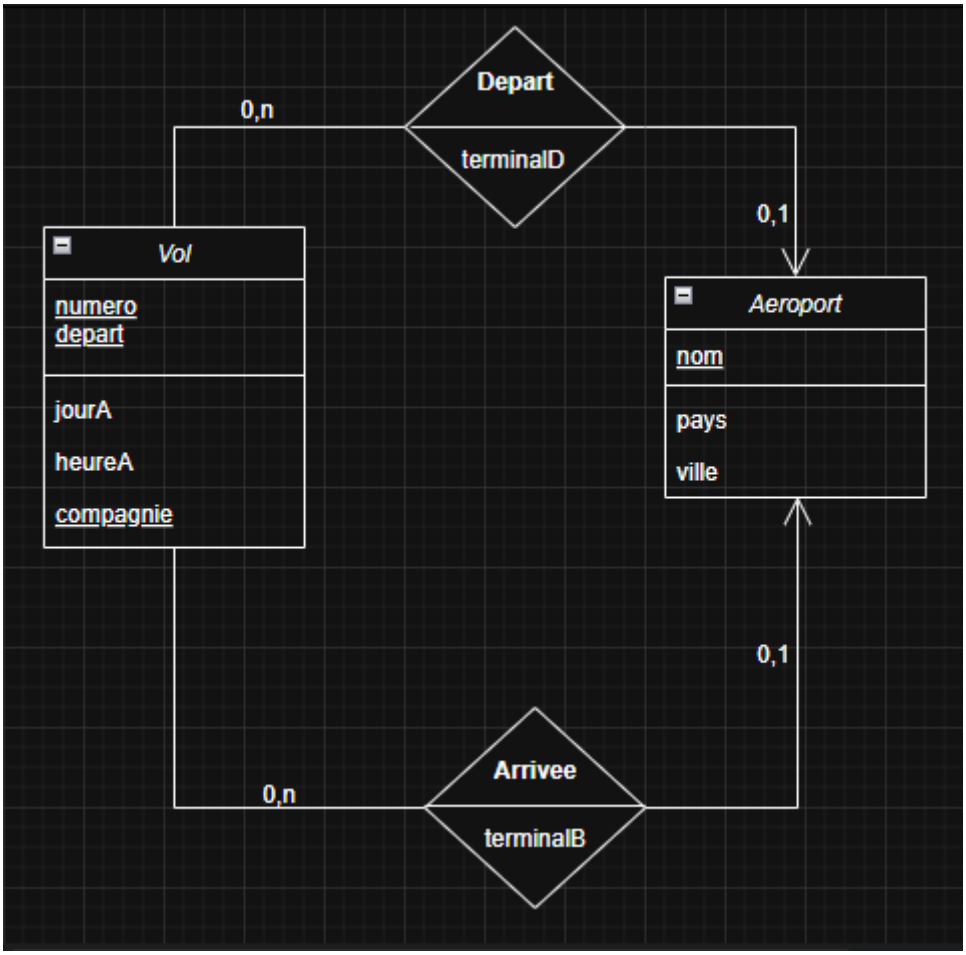
Annexe 1 : Méthode appelant l'API pour en récupérer la liste des aéroports

```
// Récupère les vols faisant office de premier vol d'une suite de correspondances
static List<Correspondance> _recupererDeparts(List<Vol> vols) {
  final correspondances = <Correspondance>[];

  for (final vol in vols) {
    final aUnVolPrecedent = vols.any((autre) =>
      autre.codeAeroportA == vol.codeAeroportD &&
      autre.tempsA.isBefore(vol.tempsD)
    );

    if (!aUnVolPrecedent) {
      correspondances.add(Correspondance.fromVol(vol));
    }
  }
  return correspondances;
}

// Fonction récursive récupérant les correspondances suivantes
static void _suiteVolsCorrespondances(Correspondance correspondance, List<Vol> lesVols, List<Correspondance> lesCorrespondances) {
  final volsSuivants = lesVols.where((vol) {return vol.codeAeroportD == correspondance.codeAeroportA && vol.tempsD.isAfter(correspondance.tempsA);}).toList();

  if (volsSuivants.isEmpty) {
    lesCorrespondances.add(correspondance);
  }
  else {
    for (final volSuivant in volsSuivants) {
      final newCorrespondance = Correspondance.fromAutreCorrespondance(correspondance);
      newCorrespondance._ajouterVol(volSuivant);
      _suiteVolsCorrespondances(newCorrespondance, lesVols, lesCorrespondances);
    }
  }
}

static List<Correspondance> genererCorrespondances(List<Vol> vols) {
  final correspondances = <Correspondance>[];
  final departs = _recupererDeparts(vols);

  for (final correspondance in departs) {
    _suiteVolsCorrespondances(correspondance, vols, correspondances);
  }

  return correspondances;
```

Annexe 2 : Suite de méthodes statiques dans la classe Correspondance permettant de lier des vols entre eux pour créer une liste de correspondances.



Annexe 3 : La liste des vols avec correspondances, chaque « page » est accessible facilement depuis le menu à gauche, les informations de chaque vols sont affichées dans la liste et en cliquant sur un vol avec correspondances il est possible d'obtenir la liste des vols qui composent le voyage.



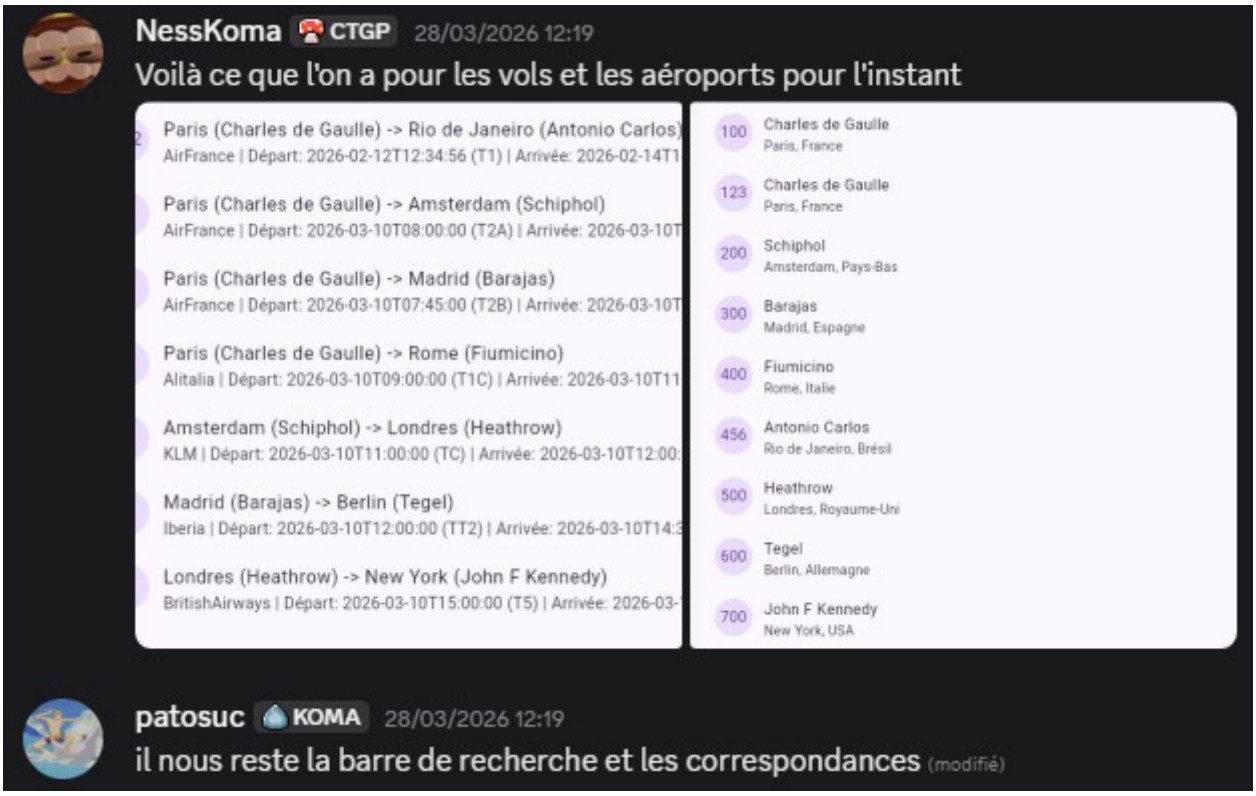
Annexe 4 : Modèle MCD de notre base de données

```sql
/*
c - En considérant les horaires des vols, veuillez fournir la liste des villes accessibles
depuis Paris avec un vol comprenant DEUX correspondances.
*/
select aeroFin.ville from AEROPORT aeroFin
join VOL vol3 on aeroFin.code = vol3.codeAeroportA
join AEROPORT aeroMid2 on aeroMid2.code = vol3.codeAeroportD
join VOL vol2 on aeroMid2.code = vol2.codeAeroportA
join AEROPORT aeroMid1 on aeroMid1.code = vol2.codeAeroportD
join VOL vol1 on aeroMid1.code = vol1.codeAeroportA
join AEROPORT aeroDep on aeroDep.code = vol1.codeAeroportD
where aeroDep.ville = 'Paris' and vol2.tempsD >= vol1.tempsA and vol3.tempsD >= vol2.tempsA;

-- d - Veuillez fournir la liste des villes accessibles depuis Paris, en tenant compte des horaires de vol, avec des vols directs ou un nombre quelconque de correspondances.
WITH aeroDebut(code, ville) AS (
    SELECT aero2.code, aero2.ville FROM AEROPORT aero2
    JOIN VOL ON aero2.code = VOL.codeAeroportA
    JOIN AEROPORT aeroParis ON aeroParis.code = VOL.codeAeroportD
    WHERE aeroParis.ville = 'Paris'
    UNION ALL
    SELECT aeroFin.code, aeroFin.ville FROM aeroDebut
    JOIN VOL ON aeroDebut.code = VOL.codeAeroportD
    JOIN AEROPORT aeroFin ON aeroFin.code = VOL.codeAeroportA
)
CYCLE code SET cycle TO 1 DEFAULT 0
SELECT DISTINCT ville FROM aeroDebut;
```

Annexe 5 : Des requêtes pour notre base de données permettant de récupérer les villes accessibles depuis Paris grâce à des vols avec 2 correspondances, et pour récupérer les villes accessibles depuis Paris grâce à des vols directs ou indirects, utilisant la récursivité.

Branch	Updated	Check status	Behind	Ahead	Pull request
SPA	14 minutes ago		2	31	
flutter	2 hours ago		2	45	
develop	4 days ago		2	35	
Flask_restx	last week		2	30	#5
modele_relationel	last week		2	12	#3
modele_objet-relationnel	3 weeks ago		2	14	#1

Annexe 6 : La liste des branches utilisées sur notre projet



Annexe 7 : Une discussion sur le groupe Discord dédié au projet permettant d'indiquer l'avancé de celui-ci